National Information Technology Development Agency

Folio Developer Guide

Architecture, components, data model, service contracts and development

National Information Technology Development Agency · Office of the Director-General/CEO · DGO Digital Operations

Document	Folio Developer Guide
Reference	NITDA-FOLIO-HO-2.4
Version	1.0
Date	20 October 2026
Classification	Internal
Built from	Repository dgoeaa/ECM_DOCS_DEV, commit c6e49d73
Build	v2026.10.20-1 (Workspace and Portal packages)
Register of record	Version 1.12.87: 0 open, 122 closed

Contents

2 · Architecture 4

2.1 The shape of the system 4

2.2 The surfaces 5

2.3 Identity: sign-in by e-mailed code 5

2.4 Authorisation: roles, personas and data scope 7

2.5 Configuration and delivery 7

2.6 The standard reply envelope 8

2.7 Observability 8

2.8 Limits that shape the design 9

2.9 Related records 9

3 · Components 10

3.1 Folio Workspace: how the code is layered 10

3.2 Core services 10

3.3 Screens (31 routes, 13 workspaces) 12

3.4 The two deliberate exceptions to "everything through the data client" 13

3.5 NITDA Document Portal (document-portal/) 13

3.6 DGO Correspondence Mailer (mailer/) 14

3.7 Platform-team tools 14

3.8 Power Automate flows 14

3.9 DYNAMIC_ACTIONS operations 15

4 · Data model 16

4.1 Sites 16

4.2 Workspace lists 16

4.3 Portal lists 18

4.4 Operational lists (all flows) 19

4.5 Identifiers 19

4.6 The Workspace's working copy (client state) 20

4.7 Roles, personas and statuses 20

4.8 Status vocabularies 21

6 · Service contracts 23

6.1 A request 23

6.2 The client's contracts 23

6.3 Status codes a service returns 25

6.4 Portal and Mailer services 25

6.5 Where to look next 25

7 · Developer guide 27

7.1 Ground rules 27

7.2 Set up 27

7.3 Run it 28

7.4 Check it 28

7.5 Change a screen 28

7.6 Change a flow 29

7.7 Build and deliver the Workspace 30

7.8 Add a contract key (a new service) 30

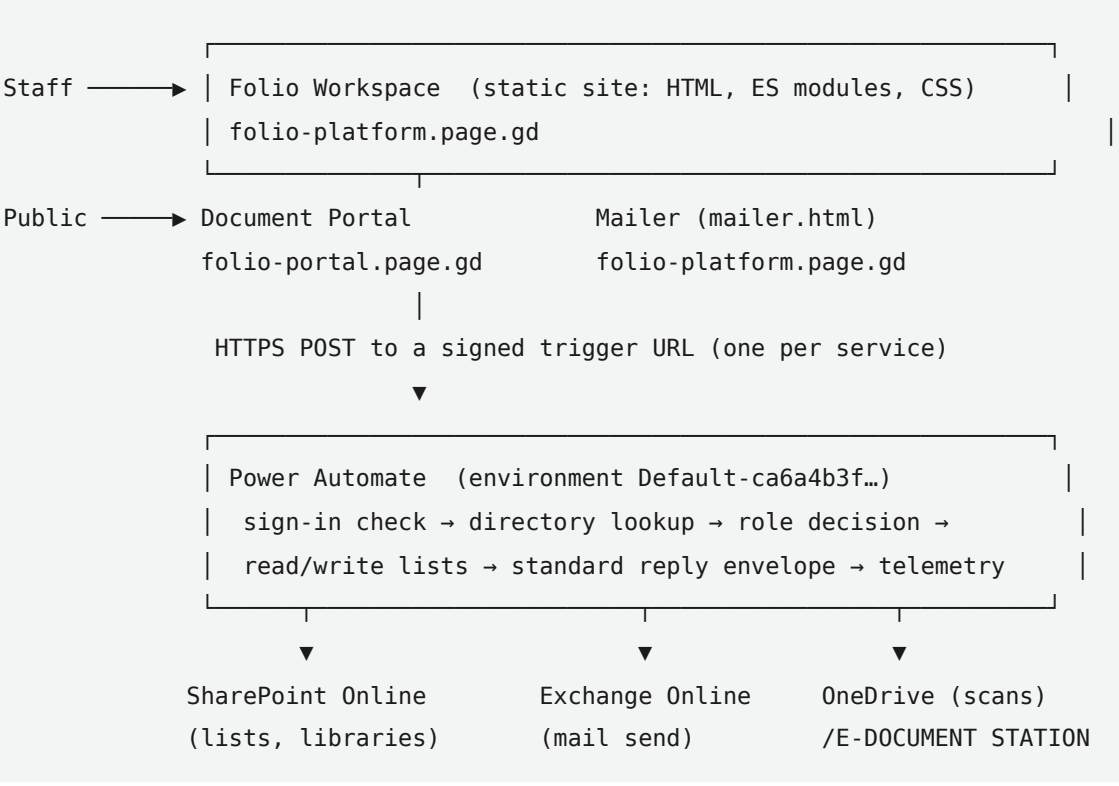
7.9 Generated files 31

7.10 Debugging a live problem 31

2 · Architecture

2.1 The shape of the system

Folio is a set of **static browser applications** that call **Power Automate flows** directly. The flows read and write **SharePoint lists**, and send mail through **Exchange Online**. There is no application server, no database server and no API gateway of Folio's own.



Three decisions explain most of the design:

1. **The browser is only a client.** Every rule that matters (who the caller is, what their role allows, what data they may see) is decided **inside the flow**. Route guards in the browser only shape what people see; they do not protect data.
2. **A trigger URL is a credential.** Possession of a signed URL lets anyone call that flow. The flow therefore never trusts the request body for identity; it takes the caller from the sign-in proof (§2.3).
3. **Zero build, zero runtime dependencies.** The Workspace runs as written: native ES modules, no bundler, no framework. What the repository holds is what the browser runs, apart from one generated file, `config/config.local.js`, which carries the signed addresses (§2.5).

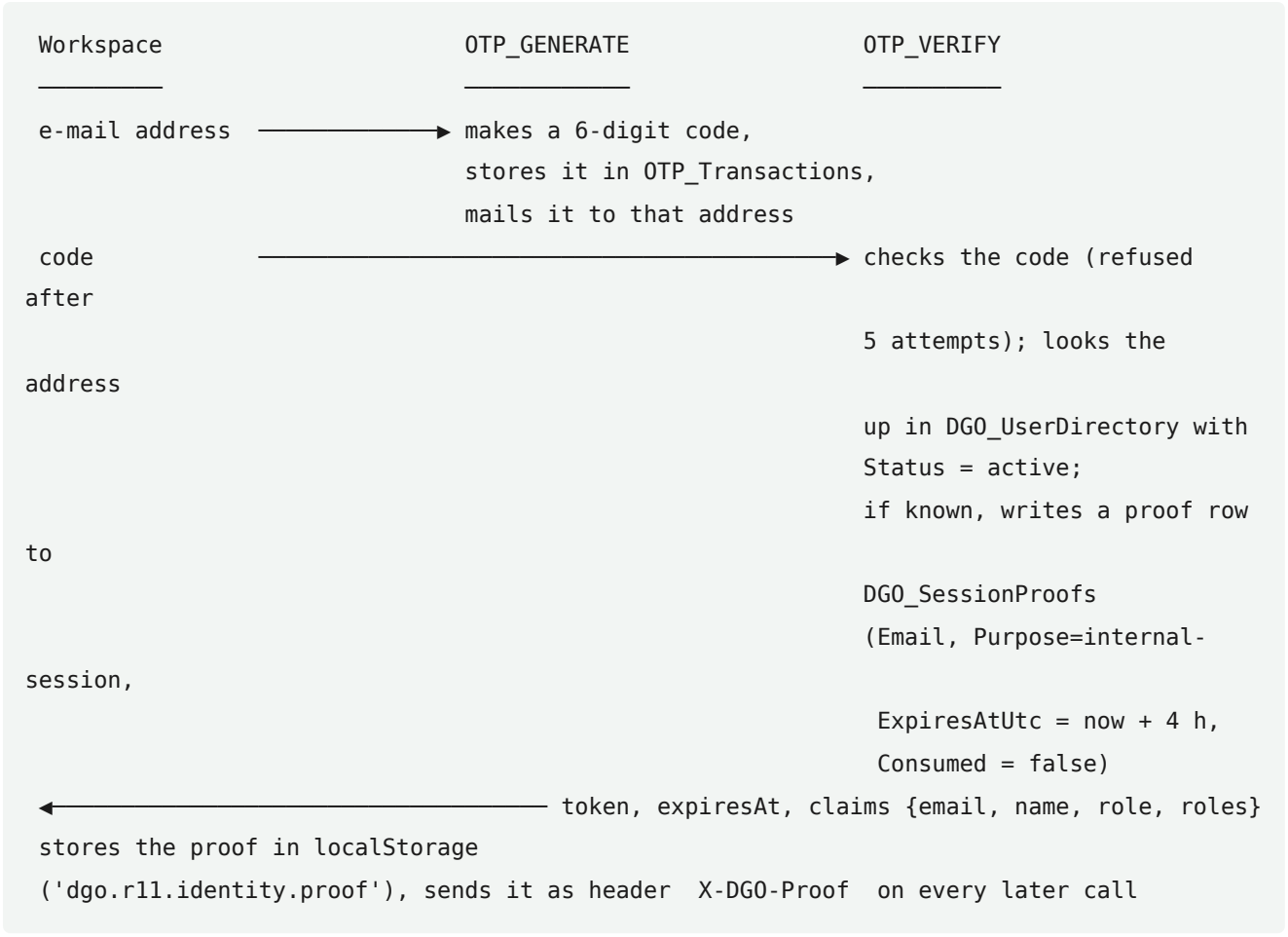
2.2 The surfaces

Surface	Users	Hosted at	Source in this repository
Folio Workspace	Staff: registry, officers, directors, executives, administrators	`https://folio-platform.page.gd/` (primary host, owner decision 2026-10-08)	repository root: `index.html`, `config/`, `core/`, `modules/`, `shared/`, `styles/`, `assets/`
NITDA Document Portal	The public: submit correspondence, track it, request support	`https://folio-portal.page.gd/` (owner, 2026-10-19)	`document-portal/`
DGO Correspondence Mailer	DGO staff sending official letters	`https://folio-platform.page.gd/mailer.html`, one file beside the Workspace (owner, 2026-10-19)	`mailer/`
Folio Estate Console	Platform team	opened from a file	`tools/admin-console.html`
Folio Flow Workbench	Platform team	local only (127.0.0.1)	`tools/flow-workbench/`
Folio Vault	Platform team	local, optional	`tools/flow-capsule/`
Folio Central Administration Centre	Platform team	local only	`tools/admin-centre/`

Each runs on its own port during development: `npm run platforms` lists them (see the Developer guide).

2.3 Identity: sign-in by e-mailed code

There is no external identity provider in the request path. Identity is a **proof** issued after a one-time code:



Every other service does the same check first (as built in FETCH_ALL, for example):

1. Read `IDENTITY_PROOF_REQUIRED` from `Flow Configuration` (it is `true` in production).
2. Take the token from `X-DGO-Proof` (or `Authorization: Bearer …`).
3. Look it up in `DGO_SessionProofs`: same token, `Consumed eq 0`, `Purpose eq 'internal-session'`, and `ExpiresAtUtc` later than now.
4. The caller is the proof row's `Email`, never anything in the request body.
5. Look the caller up in `DGO_UserDirectory` (`Status eq 'active'`) to get their **Role** and **Persona**. Not found or not active: **401**.

Facts that follow from this (read from the live flows on 2026-10-10):

- OTP_GENERATE mails a code only to an **active** `DGO_UserDirectory` member (2026-10-10). Its reply is the same for any address, so the sign-in box does not reveal who is a user. OTP_VERIFY checks the directory again before it issues a proof.
- A proof lasts **4 hours**. After that, the next call answers 401 and the Workspace asks for a new code.
- The proof lives in `localStorage` (owner, 2026-10-20), so every tab of the Workspace in the same browser shares one sign-in, and it survives a reload and a browser restart until its 4 hours run out. **Sign out** (the account menu) ends it in every tab at once; a sign-in or sign-out in one tab reaches the others through the browser's `storage` event.

- Revoking access is immediate: set the person's `Status` to anything but `active`, and their next call answers 401.
- The authentication contract, with its numbered obligations, is in `docs/architecture/AUTHENTICATION_CONTRACT.md`.

2.4 Authorisation: roles, personas and data scope

Three layers, decided in this order:

Layer	Decided by	Effect
Active directory entry	the flow, from `DGO_UserDirectory.Status`	Not active: 401 for everything.
Data scope	the flow (FETCH_ALL `Compose_FetchAll_Scope`)	**all**: systemAdmin, userAdmin, executive, operator, or persona admin, executive or registry. **directorate**: director (today their own items only, ITEM-91). **own**: everyone else, who receives only their own tasks, those tasks' documents and comments, and only their own e-mail address in the directory.
Screens	the browser (`config/rbac.config.js` `RoleRouteAccess`)	Which menu entries a role sees. This is presentation; the flow has already decided the data.

The six roles, their personas and permissions are listed in the Data model.

2.5 Configuration and delivery

What	Where	Who changes it
Signed service addresses	`config/config.local.js` in the deployed site (generated, never committed)	`npm run config:rebuild`, which reads them live from the tenant
Digest of what was last delivered	`docs/folio-Master-Flows-and-Endpoints/registers/config-build-ledger.json` (digests only, no URL)	written by `config:rebuild`; compared by `config:drift`
Sign-in enforcement	`window.DGO_CONFIG.auth = { enabled: true, roleSource: "verified" }` in `config.local.js`	the build
Browser caching	`.htaccess` at the site root: `no-cache` for HTML, JS, CSS and JSON	the build (`scripts/package.mjs` includes it)

What	Where	Who changes it
Server-side switches	SharePoint `Flow Configuration` list (`Title` = key, `ConfigValue` = value), for example `IDENTITY_PROOF_REQUIRED` , `ALLOWED_ORIGIN…` , `DATASET_…_SITE_URL` and `…_LIST_ID` , `ENVIRONMENT_CODE`	administrators with list access
Flow logic	released packages in `docs/folio-Master-Flows-and-Endpoints/packages/` , applied with `npm run flows:update`	platform team, through a release (`npm run idrelease`)

A delivered Workspace build is one zip, uploaded to the site root. Because the addresses are inside it, **the zip is private**.

2.6 The standard reply envelope

Every Folio service answers in the same shape:

```
{
  "ok": true,
  "status":  { "http": 200, "code": "OK", "message": "Success" },
  "request": { "requestId": "…", "trackingId": "", "action": "fetchAll", "operation":
"read", "source": "…" },
  "timing":  { "receivedAtUtc": "…", "completedAtUtc": "…", "durationMs": 0 },
  "data":    { },
  "errors":  [ ],
  "meta":    { "ts": "…", "runId": "…", "flowName": "…", "contractVersion": "2026-08-21.1"
}
}
```

`core/contracts.js` unwraps it on the client. A refusal keeps the shape, with `ok: false`, an HTTP status of 401, 403, 404, 409, 429 or 500, and the reason in `errors` .

2.7 Observability

Signal	Where
Every run of every flow	Power Automate run history (kept 28 days)
A record of each run, with secrets masked	SharePoint `Portal Flow Telemetry` , written by each flow's `Scope_Flow_Data_Capture`
Mail sent by a flow	`Portal Outbox Receipts`
Governed changes	`DGO_AuditLog` , plus audit columns on the records themselves

Signal	Where
In the browser	**System Health** (`#/diagnostics`): runtime checks, collections, the endpoint table and probes

2.8 Limits that shape the design

Limit	Value	Consequence
Actions in one flow	500	`USER_ADMIN` became its own flow when DYNAMIC_ACTIONS would have reached 524. `flows:update` refuses a package over the limit before writing.
Characters in one expression	8,192	`flows:update` refuses a package with a longer expression.
Indexed columns per list	20	Global Tracking Queue is full, which is why directors' directorate view waits on ITEM-91.
Items read per call	FETCH_ALL reads the newest 500 documents and 500 tasks	Older records are reached through Lookup.
Proof lifetime	4 hours	People sign in again after 4 hours.
FETCH_ALL reply	about 6.4 MB, 15 to 25 seconds, for an "all" caller	The first screen fills about 20 seconds after sign-in.

2.9 Related records

- Estate description (generated): `docs/PLATFORM.md`
- Target architecture: `docs/architecture/TARGET_ARCHITECTURE.md`
- Access specification: `docs/architecture/PLATFORM_ACCESS_SPEC.md`
- Every flow and service record: `docs/folio-Master-Flows-and-Endpoints/`

3 · Components

3.1 Folio Workspace: how the code is layered

```
index.html                entry page: boot screen, 15-second start-up watchdog (pauses
for sign-in)
.htaccess                 browser caching rules for the deployed site
config/                   configuration as ES modules: routes, roles, service contracts,
policies
  config.local.js         generated per deployment: signed addresses + auth switch (never
committed)
core/                     the runtime: state, sign-in, data access, domain rules, audit
shared/                   the shell (navigation, header, toasts), dialogs and shared UI
runtimes
modules/                  one file per screen (route id = file name)
styles/  assets/          design system CSS, fonts, icons, brand images
```

Rules the code follows (conventions, read from the code; no single check enforces them all):

- A **module** draws one screen and calls **core** services. It does not call `fetch` for business data itself; everything goes through `core/data-client.js`. The two exceptions are the Admin Suite's request composer and the Endpoint Console's probe, which call a service without a sign-in proof on purpose (§3.4).
- **config/** holds data, not behaviour. A rule that changes per role, status or category lives there.
- **core/boot.js** is the composition root: it is the only core file that imports the screens (to register their routes). Other core files do not import from **modules/**.

Drawn diagrams of these layers, and of trust zones and the lifecycle, are in `docs/architecture/components.html` and `docs/architecture/architecture.html`.

3.2 Core services

File	Responsibility
`core/boot.js`	Start-up: sign-in first (when enforced), register routes, draw the shell, resolve the deep link, then start the data load (`loadRuntimeData`).
`core/router.js`	Hash routes (`#/home` , `#/lookup` , …); asks `config/rbac.config.js` whether the role may open a route.
`core/state.js`	The single client state object. Persists to `localStorage` (`dgo.r11.viewport.runtime.state`), except `runtime` , which is never persisted. Signed addresses are never persisted.

File	Responsibility
`core/auth.js`	The sign-in gate for every governed call: token cache, `ensureAuthenticated`, `authHeaders`, the verified role.
`core/otp-identity.js`	The e-mailed-code provider: request a code, submit it, hold the proof in `localStorage` (`dgo.r11.identity.proof`), shared by every tab, follow a sign-in or sign-out made in another tab, sign out everywhere.
`config/auth.config.js`	Sign-in settings: provider `otp`, header `X-DGO-Proof`, renewal 120 s before expiry. Off by default; the deployed `config.local.js` turns it on.
`core/endpoint-registry.js`	Resolves a contract key to its address: deployment manifest, then an audited operator override, then the packaged address.
`core/data-client.js`	The only path to a service: sign-in check, address, confirmation for writes, a correlation ID, timeout and retry, the proof header, the envelope.
`core/server-writes.js`, `core/server-payloads.js`	Every write a screen records on the server (task updates, document state, minutes, approvals and decisions, briefs, meetings, projects, reminders, new records, registry files, audit events), built for the running Dynamic Actions package.
`core/fetch-manager.js`	De-duplicates identical requests and applies the read cache.
`core/cache-manager.js`	In-memory read cache (FETCH_ALL: 10 minutes). Cleared on reload.
`core/data-loader.js`	`loadRuntimeData()`: calls FETCH_ALL (falls back to FETCH_ACTIVITIES), maps the reply into state, records `runtime.lastLoad`. `requestSync()` is behind every refresh button.
`core/contracts.js`	Unwraps the standard reply envelope.
`core/flow-confirmation.js`	Shows a confirmation before any write or operation call.
`core/domain.js`, `core/entity-store.js`	Business rules: status from `Progress`, normalising documents and tasks, directorate filtering for display.
`core/audit-log.js`	In-browser audit trail of governed actions (sign-in events, loads, writes).
`core/pending-queue.js`, `core/offline-action-queue.js`	Writes that failed for want of a connection, kept for retry. A write the server refused (4xx) is shown, not kept, and an entry pushed out of the 250-entry queue is recorded in the audit trail.
`core/deeplink-resolver.js`	Opens the right screen and record from an e-mailed link (`?DGOPID=…`).
`core/scan-intake-service.js`	Lodging a scanned document through SCAN_INTAKE.
`shared/shell.js`	The frame: navigation by role, header buttons (search, help, ⟳ refresh, notifications, profile), toasts.

File	Responsibility
`shared/sign-in-dialog.js`	The "Sign in to continue" dialog.

3.3 Screens (31 routes, 13 workspaces)

The menu holds 13 workspaces, one per stage of the work (`config/workflow-clarity.config.js` , module rationalisation Phase 2, 2026-10-20). Each workspace is a set of tabs, and each tab is one route: every route keeps its own address, role rule and action owner. A role sees a workspace when it may open at least one of its tabs; the menu entry opens the first.

Group	Workspace	Tabs (route)	Purpose
START HERE	Command Center	Overview (`home`) · ERP–ECM Charter (`ecm-erp-charter`)	What needs attention now; the ERP–ECM boundary as reference.
OPERATIONS	Intake	To triage (`correspondence`) · Counter deposit (`scan-intake`) · Routed (`activities`) · Assignment Desk (`single-assignment`) · Bulk assignment (`bulk-assignment`)	Every incoming record in one list: triage, decide, assign, or route to SIWES, NYSC or archive.
	My Work	My tasks (`orchestrator`) · Acknowledge (`acknowledgment`) · Comments (`comments`)	Acknowledge, start, update and complete assigned work (Mine, Team, Overdue, Escalated).
	Registry	File custody (`registry`)	File jackets: open, route, receive and close; every move recorded.
	Lookup	Lookup & Direct Action (`lookup`)	Find any record by reference, sender or subject, and act on it.
CONTROL	SLA & Tracking	Monitoring (`response-tracking`) · SLA risk (`fasttrack`)	Responses and ageing; work at risk of missing its target.
	Decisions	Approvals (`approvals`) · DG/CEO (`executive`) · Briefs (`briefs`)	Everything waiting for a decision.
	Meetings	Meetings (`meetings`)	Request and record meetings; agreed actions are recorded for their owners.
	Projects	Projects (`projects`)	Projects and the measures they are tracked against.
	Insight	Statistics (`statistics`) · Reports (`reports`)	Trends, and the formal M1–M10 reports from one report engine.
CLOSURE	Outbound	Send by email (`correspondence-email`) · Record a dispatch (`dispatch`) · Close & archive (`archive`)	Outward correspondence, dispatch records and closure with evidence.

Group	Workspace	Tabs (route)	Purpose
SYSTEM	Admin Suite	Suite (`admin-suite`) · Administration (`settings`) · Health (`diagnostics`) · Runtime (`operator-hud`) · Endpoints (`endpoint-console`) · Users (`user-admin`)	Monitoring, control and management.
	Assistant	Assistant (`assistant`)	Guidance on the operating model.

What each screen records on the server, and how, is in `core/server-writes.js`: every write is built in the shape the running Dynamic Actions package accepts, and `tests/server-writes.test.mjs` runs each through the flow simulator.

Each route's code is `modules/<route>.js`. Who may open which route is in the Data model.

3.4 The two deliberate exceptions to "everything through the data client"

Screen	What it sends	Why
Endpoint Console probe (`modules/endpoint-console.js`)	`{"action":"<contract action>"}` to every read service, **with no sign-in proof**	It tests that each address is reachable and answers JSON. With sign-in enforced, the expected answer is **401**. A 401 there means the service is alive and protecting itself, not that it is broken.
Admin Suite request composer (`modules/admin-suite.js`)	A body an administrator writes, checked against the trigger schema	For administrators testing a service by hand. Also carries no proof.

3.5 NITDA Document Portal (`document-portal/`)

A public progressive web app (service worker, manifest, offline shell), deployed under `/CG/`.

Page	Purpose
`index.html`	Landing page
`submit.html`	Submit correspondence with attachments (e-mail verification first)
`track.html`	Track a submission by reference and e-mail
`support.html`	Raise a support case
`404.html`	Not found

Its seven services: `SUBMISSION`, `UPLOAD`, `STATUS`, `SUPPORT`, `VERIFY`, `VERIFY_CONFIRM` and `WRITEBACK`. The first six are public and rate-limited per IP address (20, 60, 30, 10, 5 and 5 per hour respectively). `WRITEBACK` is how Folio sets the status a submitter sees (20 per hour). Its records live in the `Portal …` lists (Data model §4.3).

3.6 DGO Correspondence Mailer (`mailer/`)

A single-page app that sends one personalised official letter per addressee, each with its own signed attachment, and logs every send. Deployed at `/mailer.html`. Its seven services: `MAIL_SEND`, `MAIL_LOG`, `MAIL_DIRECTORY`, `MAIL_TEMPLATES`, `MAIL_PROVISION`, `MAIL_AUTH` and `MAIL_DATA`. **Outbound letters are sent only through the Mailer**: Folio's Dispatch screen records that a dispatch happened; it does not send mail itself (owner decision on ITEM-25, 2026-10-08). See `mailer/README.md`.

3.7 Platform-team tools

Tool	What it is for	Start
Folio Estate Console (`tools/admin-console.html`)	One-file console over every service address and the flow catalogue	`npm run start:console`
Folio Flow Workbench (`tools/flow-workbench/`)	Inspect, compose, validate and send requests to catalogued flows	`npm run start:workbench` (Python 3.7+, binds 127.0.0.1)
Folio Vault (`tools/flow-capsule/`)	SQLite registry holding each signed URL behind an alias it never returns	`npm run start:capsule` (Python 3.11+)
Folio Central Administration Centre (`tools/admin-centre/`)	Monitors every platform; alerts, incidents, changes, a hash-chained audit trail	`npm run start:centre` (Python 3.9+, loopback only); see `docs/admin-centre/`
Platform Hub	The front door listing every platform and whether it is up	`node scripts/run-platform.mjs hub`

3.8 Power Automate flows

The Workspace's 15 delivered addresses resolve to 13 flows. The table is taken from the build ledger (`registers/config-build-ledger.json`), which is written from a live read of the tenant.

Contract key	Flow	Reads or writes
`FETCH_ALL`	IP_FETCH_ALL_BOOTSTRAP	Reads the Workspace's data, scoped by role
`FETCH_ACTIVITIES`, `GET_DOCS`	IP_Get_Docs_Endpoint	Reads documents (FETCH_ALL's fallback)
`REFERENCE_DATA`	IP_Reference_Data_Endpoint	Reads categories, departments, lookups
`FETCH_EMAIL_ATTACHMENTS`	IP_Retrieve_Email_Attachment_Endpoint	Reads attachments
`SINGLE_ASSIGNMENT`	Single_Task_Assignment	Creates a task for one assignee
`BULK_ASSIGNMENT`	BULK_OPS_DATA_RETRIEVAL_HTTP	Creates tasks in bulk

Contract key	Flow	Reads or writes
`DYNAMIC_ACTIONS`, `SUBSIDIARY_ACTIONS`	IP_Dynamic_Global_Actions_Endpoint	Task and record operations (§3.9)
`USER_ADMIN`	IP_User_Admin_Endpoint	Adds and changes people in the directory
`EMAIL`	IP_SEND_EMAIL	Sends an e-mail from the Workspace
`EMAIL_RELATED_TASK`	IP_Create__Task__for_received_Email_Endpoint	Creates a task from an e-mail
`OTP_GENERATE`	IP_OTP_GENERATE	Mails a sign-in code
`OTP_VERIFY`	IP_OTP_VERIFY	Exchanges a code for a proof
`SCAN_INTAKE`	IP_SCAN_INTAKE	Lodges a scanned document from the registry desk

Contract keys the client knows but the go-live build leaves empty: `AI_CHAT`, `AI_DOC_ANALYSIS` and `AI_EMAIL_ANALYSIS` (ITEM-88), and `DISPATCH_OUTBOUND` and `ARCHIVE_REFERENCE`, which are served as operations of DYNAMIC_ACTIONS instead.

Scheduled and event flows with no address: `Folio_Incoming_Correspondence_Scan_Processing_Flow` (watches `/E-DOCUMENT STATION`, every minute), `AUTO_SCHEDULED_SWEEP`, the Portal daily sweep, and the GOV provisioning flows.

Every flow record (definitions, packages, captures, evidence) is in `docs/folio-Master-Flows-and-Endpoints/`. A per-flow description, including FETCH_ALL action by action, is in its `provisioning/flows/` folder.

3.9 DYNAMIC_ACTIONS operations

The operations the released package recognises (`createArray(…)` in its operation switch): `update`, `update-task`, `transitionstatus`, `logauditevent`, `register`, `upsert-record`, `open-file`, `move-file`, `link`, `void`, `reparent` and `merge`.

Mapped onto those: **dispatch** and **dispatch-outbound** set the document's Status to *Treated*; **archive-reference** sets it to *Treated* and its IdState to *ARCHIVED* (ITEM-25, proven live 2026-10-08).

Still answering **501** (ITEM-25, decision E1 in the walkthrough): `activity-archive`, `activity-siwes` and `activity-nysc`.

4 · Data model

All business data lives in **SharePoint Online lists** on four sites. The browser holds a working copy for the session only. This chapter names every list Folio's adopted flows touch (25, read from 74 flow definitions by `scripts/flow-list-sweep.mjs` into `docs/folio-Master-Flows-and-Endpoints/registers/flow-list-map.json`), what each is for, and the columns that matter.

> **Column names are internal names.** SharePoint shows display names in the browser, but flows read and write **internal names** (for example `CC_x0027_dTo` for "CC'd To"). The complete, verified column lists are in `docs/deployment/RUNBOOK.md` §1.2 and §1.4. Use those when writing a flow.

4.1 Sites

Site (`https://nitdanigeria.sharepoint.com/sites/…`)	Holds
`NITDADGO-EAAACTIVITYTRACKING`	The operational records: documents, tasks, comments, categories, departments, Flow Configuration, sign-in codes
`DGO_ECM_GOVERNANCE`	People, roles, sign-in proofs, the audit log
`NEDMS`	The Portal's submissions, attachments, status timeline, upload tickets; the Central Registry
`Global_Digital_Documents_Centre`	The Portal's verification, rate limits, sequence counters, support cases; telemetry and mail receipts for all flows

4.2 Workspace lists

List	Site	What one row is	Key columns
DGO DIGITAL OPS (`1f1cb303…`)	activity tracking	A document or piece of correspondence (a *reference*)	`Title` , `RefIDD` (reference), `Category0` , `Status` , `OverallStatus` , `AssignedTo` , `RoutedToDSU` , `CC_x0027_dTo` , `StartDate` , `BodyExtract` , plus identity columns `EntityKey` , `ReferenceId` , `IdState`

List	Site	What one row is	Key columns
Global Tracking Queue (`ee82725a…`)	activity tracking	A task assigned to an officer	`Title` , `RefIDD` (the document), `Reference_ID` , `AssignedTo` (indexed), `AssignedBy` , `GDSUROUT` (the unit routed to), `DueDate` , `Priority` , `Progress` (the task's status), `TaskType` , `Acknowledge_x0020_Task` , `AcknowledgedBy` , plus `TaskRef` , `EntityKey` , `ParentKey` , `RootKey` , `IdState` , `Rev`
Task_Comments (`1932d687…`)	activity tracking	A comment on a document or task	`Title` , `RefIDD` , `Description` , `Created` , `EntityKey` , `AuthorKey`
Organizational_Categories_Matrix (`f64c8b65…`)	activity tracking	A correspondence category and its default routing	`Category` , `Subcategory` , `DSU_KEY` , `Priority` , `Timeline` , default primary and supporting units, units to inform
Organizaitonal_Departments_Information (`8aab9c4e…`)	activity tracking	A department or unit (spelling as in the tenant)	`Title` , `DSU_KEY` , `DSU_Email` , `DSU_HeadEmail` , `DSU_HeadTitle`
Flow Configuration (`9bc168c3…`)	activity tracking	One server-side setting	`Title` (the key), `ConfigValue`
OTP_Transactions (`9421d473…`)	activity tracking	A sign-in code issued to an address	`Title` (address), `OTP_Code` , `Expires_At` , `Is_Verified`
DGO_UserDirectory (`3d591f5b…`)	governance	A person who may use Folio	`Email` , `FullName` , `Directorate` , `Department` , `JobTitle` , `Role` , `Persona` , `Status` , `AccessScope` , `PilotCohort` , `DisabledReason` , `UserId`
DGO_RoleCatalogue (`f675598b…`)	governance	A role definition	`RoleId` , `Persona` , `PermissionsJson` , `AllowedRoutesJson` , `Active` , `Version`
DGO_SessionProofs (`444102d6…`)	governance	A sign-in proof	`Title` (the token), `Email` , `Purpose` (`internal-session`), `ExpiresAtUtc` , `CreatedAtUtc` , `Consumed`
DGO_AuditLog (`be0c7af1…`)	governance	A governed change	written by DYNAMIC_ACTIONS and USER_ADMIN

List	Site	What one row is	Key columns
DGO_UserRoleHistory (`9fe1872f…`)	governance	A role change	written on role assignment
NITDA_Central_Registry (`f553959f…`)	NEDMS	A registry file	`Title`

Two facts that catch people out:

- **Global Tracking Queue has no `Status` column.** A task's status is `Progress` (`core/domain.js`). A document's status is `DGO DIGITAL OPS/Status` . They are different lists and different columns.
- **Global Tracking Queue uses all 20 indexed columns SharePoint allows.** Filters that must be fast use indexed columns (`AssignedTo` for "my tasks"). Adding a filter on an unindexed column to a list this size (about 16,000 rows) gets throttled. ITEM-91 records the trade-off.

`DEV_OPS_Global_Tracking_Queue` (`22c4e38d…`) is a **test list**. Folio does not read it, and no production flow should write to it (ITEM-74).

4.3 Portal lists

List	Site	What one row is
Portal Registry	NEDMS	A public submission: reference, sender, subject, category, channel, status, submitted, updated, acknowledged and closed times
Portal Attachments	NEDMS	A file attached to a submission (declared name, stored name, size, SHA-256, link)
Portal Status Timeline	NEDMS	One status event a submitter can see (`SubmissionRef` , `AtUtc` , `Status` , `Label` , `Actor` , `Note`)
Portal Upload Tickets	NEDMS	A single-use permission to upload one file
Portal OTP Codes	Documents Centre	A Portal e-mail verification code (with attempt count)
Portal Verification Proofs	Documents Centre	A verified Portal e-mail (the Portal's equivalent of `DGO_SessionProofs`)
Portal Rate Limits	Documents Centre	A per-IP, per-service counter for a 60-minute window
Portal Sequence Counters	Documents Centre	The counter that mints reference numbers (`Prefix` , `Year` , `CurrentSequence` , `LastReferenceId` , `LockToken`)
Portal Support Cases	Documents Centre	A support request (case reference in `Title`)
Portal Audit Events	Documents Centre	A Portal audit event

4.4 Operational lists (all flows)

List	What one row is
Portal Flow Telemetry (`726c210d…`)	One flow run: flow, run ID, start, end, outcome, duration, error, and an attached run record with secrets masked
Portal Outbox Receipts (`88a81ca1…`)	One e-mail a flow sent: type, recipient, reference, time, status, attempts, last error
AI TEXT DATA LOG (`d46a2be7…`)	Text extracted by the legacy AI intake flow (retires with ITEM-80)

4.5 Identifiers

Defined by the identification system (`docs/reference/identification/IDENTIFICATION_SYSTEM.md` , catalogue version 1.2.6). The ones people meet:

Identifier	Format	On
Registry Reference	`NITDA-YYYY-NNNNN` (non-production: `ZNITDA-…`)	Correspondence
Task Reference	`TASK-YYYY-NNNNN` (`ZTASK-…`)	Tasks
Case Reference	`CASE-XXXXXX` , letters and digits without I, O, 0 or 1 (`ZCASE-…`)	Support cases
File Reference	`FILE-YYYY-NNNNN` (`ZFILE-…`)	Registry files

And the ones the system uses:

Identifier	Format	Purpose
Entity Key (`EntityKey`)	lower-case GUID v4	The permanent machine identity of every governed record. Relationships point at it; it is never shown as the reference.
Qualified Key	`<TYPE>:<entity key>` , where TYPE is one of REC, ATT, TSK, MIN, CAS, EVT, LNK, MAP, USR, ORG, BRF, MTG, PRJ, APR, REG	An entity key with its type
Identity state (`IdState`)	`ACTIVE` , `ARCHIVED` , `MERGED` , `VOID`	Whether an identifier resolves to itself (ACTIVE, ARCHIVED read-only), to another record (MERGED, see `SupersededBy`), or to nothing (VOID). It is independent of business status.
Revision (`Rev`)	positive integer	Optimistic concurrency: a write names the revision it read
Contract Key	`UPPER_SNAKE`	Names a service (`FETCH_ALL`)
Request ID / Correlation ID	GUID v4	One request; sent as `X-Correlation-Id`

All 26 identifiers, with patterns and owners, are in `identifier-catalogue.json`.

4.6 The Workspace's working copy (client state)

`core/state.js` holds one object. FETCH_ALL fills it at sign-in and on every refresh:

State key	Filled from FETCH_ALL `data.`	Holds
`activities`	`docs`	Documents (newest 500, or your own for the "own" tier)
`tracking`	`tasks`	Tasks (newest 500, or yours)
`comments`	`comments`	Comments
`users`	`users`	The directory (e-mail addresses: all for the "all" tier, only your own otherwise)
`categories`, `departments`	same names	Reference data
`portalRegister`	`register`	Public submissions (the "all" tier only)
`approvals`	`approvals`	Approvals (empty for the "own" tier)
`emails`	`emails`	Always empty in the go-live build (ITEM-74)

FETCH_ALL also returns `briefs`, `meetings`, `projects` and `files`; `core/data-loader.js` does not map those four keys into state.

Other keys (`settings`, `profile`, `pending`, `audit`, `notifications`, `dispatches`, ...) belong to the browser. Everything except `runtime` persists in `localStorage` under `dgo.r11.viewport.runtime.state`. `runtime` holds the outcome of the last load in `runtime.lastLoad` (`ok`, `key`, `counts`, `message`).

4.7 Roles, personas and statuses

From `config/rbac.config.js`. The role decides the screens and the server-side data scope.

Role	Label	Persona	Data scope (server)	Permissions
`systemAdmin`	System Administrator	admin	all	every permission
`userAdmin`	User Administrator	admin	all	user:view, create, update, disable; role:assign, role:view; audit:view
`executive`	Executive	executive	all	executive:view, executive:export, audit:view
`director`	Director / Directorate Lead	registry	directorate (own items until ITEM-91)	executive:view, route:manage, dispatch:approve, bulk:assign

Role	Label	Persona	Data scope (server)	Permissions
`operator`	Operator	registry	all	route:manage, bulk:assign
`viewer`	Read-only Viewer	general	own	none

Screens each role may open:

The menu shows each role the workspaces whose tabs it may open (Components §3.3); the screens are:

Role	Screens
systemAdmin	all
userAdmin	Command Center, Administration, User Administration, System Health, Operator HUD, Admin Suite
executive	Command Center, DGCEO Hub, Tracking & Monitoring, Acknowledgment Queue, My Work, Comments, Review & Approval, Briefs, Meetings, Projects, Reports, Statistics, Lookup, Assistant, Archive Evidence
director	Command Center, Activities, Intake & Assignment, Tracking & Monitoring, Acknowledgment Queue, My Work, Comments, Review & Approval, Dispatch, Registry Scan Intake, Briefs, Meetings, Projects, Reports, Statistics, Lookup, Assistant
operator	Command Center, Activities, Intake & Assignment, Tracking & Monitoring, Acknowledgment Queue, My Work, Assignment Desk, Bulk Assignment, Registry Scan Intake, Registry, Comments, Dispatch, Correspondence Email Desk, Meetings, Projects, Lookup, Assistant
viewer	Command Center, Tracking & Monitoring, Acknowledgment Queue, My Work, Comments, Reports, Statistics, Lookup

Directory **statuses**: `staged`, `active`, `disabled`, `unregistered`. Only `active` can sign in or receive data.

Only a **System Administrator** may grant the System Administrator role (enforced in the USER_ADMIN flow).

4.8 Status vocabularies

What a Portal submitter sees (`config/status-vocabulary.config.js`):

Key	Label	Stage	Meaning
`received`	Received	1	Logged in the registry and queued for validation
`validation`	Validation	2	Documents are being checked for completeness
`review`	Under review	3	With the assigned unit for technical assessment
`action-required`	Action required	3	Something is needed from the submitter

Key	Label	Stage	Meaning
`approved`	Approved	4	Decision issued; outcome sent to the submitter
`declined`	Declined	4	Not approved on this submission
`withdrawn`	Withdrawn	4	Closed at the request of the submitter

Internal statuses map onto these: Pending → received; Accepted and Delegated → review; Declined → declined; Archived → approved.

Documents carry `Status` on DGO DIGITAL OPS; dispatch and archive set it to *Treated*. **Tasks** carry `Progress` on Global Tracking Queue.

6 · Service contracts

How the Workspace talks to its services. The map from each contract key to its flow is in Components §3.8. Everything about an individual flow (definition, released package, live captures, evidence) is in `docs/folio-Master-Flows-and-Endpoints/`.

6.1 A request

Every call is an HTTPS `POST` to the service's signed trigger URL, made by `core/data-client.js`:

```
POST https://defaultca6a4b3f…
environment.api.powerplatform.com/powerautomate/automations/direct/…/workflows/<id>/trigge
rs/manual/paths/invoke?api-version=1&sp=…&sv=1.0&sig=<signature>
Content-Type: application/json
X-Correlation-Id: <GUID>
X-DGO-Proof: <sign-in proof>          (every service except OTP_GENERATE and OTP_VERIFY)

{
  "action": "<the contract's action, e.g. fetchAll>",
  "payload": { … },
  "requestId": "<GUID>",
  "timestamp": "<ISO time>"
}
```

- OTP_GENERATE and OTP_VERIFY are called with `flatPayload`: the fields sit at the top level (`action`, `identifier`, `otp_code`), because those flows read them there.
- The browser sends a CORS preflight first. The Power Platform gateway answers it itself, with `Access-Control-Allow-Origin: *` and the requested headers allowed (accepted as ITEM-56: the proof, not the origin, is the protection).
- The reply is the standard envelope (Architecture §2.6).

6.2 The client's contracts

From `config/endpoints.config.js` and `config/fetch-policy.config.js`. *Timeout* is the client's wait before it gives up; *retry* is how many times it tries again; *cache* is how long a read is reused.

Contract key	Action	Kind	Timeout	Retry	Cache
`FETCH_ALL`	`fetchAll`	read	90 s	1	10 min
`FETCH_ACTIVITIES`	`LIST-ACTIVITIES`	read	15 s	1	5 min

Contract key	Action	Kind	Timeout	Retry	Cache
`REFERENCE_DATA`	`lookups`	read	15 s	1	5 min
`GET_DOCS`	`getDocs`	read	15 s	1	5 min
`FETCH_EMAIL_ATTACHMENTS`	`fetchEmailAttachments`	read	15 s	1	5 min
`SINGLE_ASSIGNMENT`	`singleassignment`	write	15 s	1	
`BULK_ASSIGNMENT`	`bulkassignment`	write	15 s	1	
`DYNAMIC_ACTIONS`	`dynamicGlobalAction`	write	15 s	0	
`SUBSIDIARY_ACTIONS`	`INIT`	write	15 s	1	
`USER_ADMIN`	`userAdmin`	write	15 s	1	
`EMAIL`	`dispatchEmail`	write	15 s	1	
`EMAIL_RELATED_TASK`	`emailtotaskassignment`	write	15 s	1	
`SCAN_INTAKE`		write	15 s	1	
`OTP_GENERATE`	`generate`	identity	20 s	0	
`OTP_VERIFY`	`verify`	identity	20 s	0	
`AI_CHAT`, `AI_DOC_ANALYSIS`, `AI_EMAIL_ANALYSIS`		empty in the go-live build			
`DISPATCH_OUTBOUND`, `ARCHIVE_REFERENCE`		served as DYNAMIC_ACTIONS operations			

Writes and retries. A write with retry 1 is sent again if the first attempt times out or fails. Of the write services, only DYNAMIC_ACTIONS de-duplicates a repeated request (by request hash); the others can apply a retried write twice. This was accepted as a deferred follow-up under G-04 (obligation 6).

Every write asks the person to confirm first (`core/flow-confirmation.js`), unless the screen has already asked (it marks the request `__confirmedByUI`) or the button itself is the confirmation (sign-in passes `skipConfirmation`).

6.3 Status codes a service returns

HTTP	Envelope code	Means	The Workspace does
200	`OK`	Done	Applies the result
401	`ERR401`	No valid proof, or the person is not active in the directory	Reports the failure (a refresh shows "The registry could not be reached"). It does not clear the proof by itself. An *expired* proof never gets this far: the client knows the expiry and opens the sign-in dialog before calling.
403	`ERR403`	Signed in, but the role may not do this	Shows the refusal
404	`ERR404`	Not found (for OTP_VERIFY: the code was not accepted)	Shows the message
409	`ERR409`	Conflict: duplicate person, or the record changed since you read it (`Rev`)	Asks you to reload and retry
429	`ERR429`	Too many attempts (sign-in codes, Portal rate limits)	Asks you to wait
500	`ERR500`	The flow failed (`Scope_Catch_…`)	Shows the failure; the run record is in Portal Flow Telemetry
501	`ERR501`	The operation is recognised but not built (ITEM-25)	Shows the refusal

6.4 Portal and Mailer services

Surface	Contract keys	Notes
Document Portal	`SUBMISSION` , `UPLOAD` , `STATUS` , `SUPPORT` , `VERIFY` , `VERIFY_CONFIRM` , `WRITEBACK`	Public; per-IP limits per hour: SUBMISSION 20, UPLOAD 60, STATUS 30, SUPPORT 10, VERIFY 5, VERIFY_CONFIRM 5, WRITEBACK 20
Correspondence Mailer	`MAIL_SEND` , `MAIL_LOG` , `MAIL_DIRECTORY` , `MAIL_TEMPLATES` , `MAIL_PROVISION` , `MAIL_AUTH` , `MAIL_DATA`	Its own signed addresses, delivered as `mailer-endpoints.config.local.js`

6.5 Where to look next

Need	Record
The released package for a flow	`docs/folio-Master-Flows-and-Endpoints/packages/`

Need	Record
A flow, action by action	`docs/folio-Master-Flows-and-Endpoints/provisioning/flows/`
Which flow each key resolves to, and its workflow ID	`registers/config-build-ledger.json`, `registers/endpoint-workflow-ids.json`
Which lists each flow touches	`registers/flow-list-map.json`
What each flow runs in the tenant, by fingerprint	`registers/identification-paste-ledger.json`
The authentication obligations	`docs/architecture/AUTHENTICATION_CONTRACT.md`

7 · Developer guide

> **Status:** handbook chapter, 2026-10-10. It carries steps, and owns none: flow changes are owned by `FLOWS_UPDATE.md`, builds by `PACKAGING.md`, cutover by `RUNBOOK.md`.

For anyone changing Folio's code, configuration, flow packages or records. Read Architecture and Components first.

7.1 Ground rules

The repository's working rules are in `CLAUDE.md`. Where `CONTRIBUTING.md` differs (it still describes feature branches and pull requests), `CLAUDE.md` **governs**:

1. **One branch:** all work is committed and pushed to `claude/system-remediation-gaps-ahpmsy`. No other branch, no pull requests, never a force-push. Pull with `--rebase` before you start and before you push.
2. **Closed register items stay closed** unless new, committed evidence contradicts them.
3. **Flow and endpoint records live only in** `docs/folio-Master-Flows-and-Endpoints/`. `npm run test:masterflows` fails on one anywhere else. (This handbook links to them; it holds none.)
4. **Flow packages are frozen at a release.** Change them only for a defect found in the tenant or an owner decision, and prefer a `Flow Configuration` row to a package change.
5. **Generated files are never edited by hand.** Re-run their generator (§7.7).
6. **Secrets never enter git:** signed URLs, `config.local.js`, sign-in sessions, proofs. `npm run test:secrets` checks.

7.2 Set up

Requirements: **Node 22 or later**; Python 3.9+ only for the Workbench, Vault and Administration Centre.

```
git clone https://github.com/dgoeaa/ECM_DOCS_DEV.git
cd ECM_DOCS_DEV
git checkout claude/system-remediation-gaps-ahpmsy
npm ci
```

7.3 Run it

Command	What you get
`npm run dev`	The Workspace on `http://localhost:8080/` **with a local stand-in for every service** (`scripts/dev-server.mjs`), so screens work without the tenant. Data is kept in a local file (`DGO_DEV_DATA`).
`DGO_DEV_AUTH=1 npm run dev`	The same, with **sign-in enforced**. The dev server accepts the fixed code `000000` .
`npm start`	A plain static server: the Workspace as files, with no services behind it (demo mode without `config/config.local.js`).
`npm run platforms`	Every surface and its port. Start one with `npm run start:<id>` : `runtime` 8080, `portal` 8081, `console` 8082, `mailer` 8083, `workbench` 8765, `capsule` 8787, `centre` 8844.
`npm run setup`	Scaffolds a local `config/config.local.js` (git-ignored).

More detail: `docs/deployment/LOCAL-DEV.md` .

7.4 Check it

Command	What it runs	When
`npm run test:node`	The full gate: about 175 checks (imports, hygiene, references, secrets, generated-file drift, registers, packages, simulated flow runs)	Before every push. Run it **on the branch**: one check stamps the branch name.
`npx playwright test`	The browser suite (192 tests in 17 spec files on 2026-10-10). In a cloud session set `PLAYWRIGHT_CHROMIUM_EXECUTABLE_PATH=/opt/pw-browsers/chromium` .	Before pushing any change to screens, `core/` or `shared/` .
`DGO_BASE_URL=https://folio-platform.page.gd npx playwright test`	The same suite against a deployed site	Commissioning (MANUAL-3)
`npm run test:links`	Every relative link in the docs	After changing documents
`npm run outstanding`	Totals the open items in every register	Before choosing work

A gate failure names the command that fixes it. If it is a generated file, run that command; do not edit the file by hand.

7.5 Change a screen

1. The route id is the file name: `modules/<route>.js` exports `mount(el)` .
2. A new route needs: an entry in `config/routes.config.js` , the import in `core/boot.js` (the `modules` map), a place in `config/workflow-clarity.config.js` (visible group, or hidden with the

screen it is reached through), and a row in `config/rbac.config.js` `RoleRouteAccess` for each role that may open it.

3. Call services only through `core/data-client.js` (or `core/api.js`). Never call `fetch` for business data from a module.
4. Add or extend a browser test in `tests/*.spec.js`.

7.6 Change a flow

Flows are changed by **released packages**, not by hand in the designer.

```
edit the builder → build → simulate → release → check → apply → prove live → record
```

Step	Command	Notes
Edit	`scripts/build-identification-phase4.mjs` (internal flows) or `…-phase3.mjs` (Portal)	Each flow's package is built from these
Build	`npm run idphase4` / `npm run idphase3`	Writes `packages/identification-phase4/*.paste.json`
Simulate	`npm run test:idphase4` / `npm run test:idphase3`	Runs each package in a flow simulator against seeded lists
Release	`npm run idrelease -- --reason "…"`	Freezes every package's fingerprint in `registers/identification-release.json`. Say in your report which packages changed and why.
Check	`npm run flows:update`	Reads every live flow and compares it with its package. Writes nothing.
Apply	`npm run flows:update -- apply`	Backs up, writes, reads back. **Refuses** a package that would exceed **500 actions** or an expression longer than **8,192 characters**. Runs `config:drift` itself.
Prove	A live call against a test record, then remove the record	
Record	`npm run flows:update -- record <report>`	Files the report as evidence and updates `registers/identification-paste-ledger.json`

Sign-in for `flows:update` and `config:*`: set `FOLIO_MS_SESSION` to a saved sign-in file. Without one, it offers a code on a Microsoft page. Behind a proxy, set `NODE_USE_ENV_PROXY=1`. Only flows the account owns or co-owns can be read or written.

Full procedure: `docs/folio-Master-Flows-and-Endpoints/deployment/FLOWS_UPDATE.md`.

7.7 Build and deliver the Workspace

Step	Command
Has any address moved since the last delivered build?	`npm run config:drift` (reports `REBUILD SUGGESTED` if so)
Build the deployable files	`npm run config:rebuild -- --out <private folder> --mailer <published mailer.html>`
Commit the record of what was built	`git add docs/folio-Master-Flows-and-Endpoints/registers/config-build-ledger.json`

`config:rebuild` reads every signed address live, builds and verifies the packages, and writes, outside the repository (mode 0600):

- `FOLIO-WORKSPACE-v<version>-upload-to-site-root.zip`: upload its **contents** to the Workspace host's root, **including the hidden** `.htaccess`; it carries `mailer.html` when `--mailer` names it;
- `FOLIO-PORTAL-v<version>-upload-to-portal-site-root.zip`: the Portal with its files at the site root (`folio-portal.page.gd`); `FOLIO-PORTAL-v<version>-upload-to-site-root.zip` is the same Portal under `CG/`;
- `FOLIO-MAILER-v<version>-endpoints.config.local.js`: the Mailer's addresses.

The zips hold signed addresses: hand them over privately and delete them once deployed.

Every package is identified (`scripts/lib/package-identity.mjs`). One rebuild is one version, `YYYY.MM.DD-N` (the build date and its number that day). It appears in:

Where	What it shows
The zip's file name	`FOLIO-WORKSPACE-v2026.10.20-1-…zip`
`BUILD-ID.json` inside each site	version, package, commit, build time, `codeFingerprint` (the files without their config: the same code gives the same value in every build), `configDigest` (the config file hashed: two builds of the same code with different addresses differ here), and for the Workspace the Mailer's digest
The deployed site	Workspace: the account menu reads **Build v… · commit**; Portal: the foot of every page reads **Build v…**
`registers/config-build-ledger.json`	`version`, and per package its file name, fingerprints and file count

To check what is deployed, compare the build the site shows with the ledger, or open `/BUILD-ID.json` on the site.

7.8 Add a contract key (a new service)

This touches several records. The 2026-10-08 USER_ADMIN addition needed all of these:

1. `config/endpoints.config.js`: the contract (action, method, write, timeout).
2. The flow's package, built and simulated (§7.6), and the flow created and applied.
3. `docs/folio-Master-Flows-and-Endpoints/registers/internal-flow-register.json`: a row, and a changelog entry.
4. `docs/reference/identification/identifier-catalogue.json` `contractKeyProcess`: the process it belongs to, with a version bump and a "what changed" note in the specification.
5. `scripts/dev/endpoints.mjs`: a stand-in for the dev server, so the browser suite can call it.
6. The notification matrix: its failure-alert row.
7. Counts pinned by tests (for example the number of contract keys) and their reasons.
8. A new build (§7.7): `config:drift` reports the new key until it is delivered.

Run `npm run test:node`; each missing record fails a named check.

7.9 Generated files

Never edit these by hand. If a pull conflicts in one, take either side and re-run its generator.

File or folder	Generator
`docs/PLATFORM.md`	`npm run platform`
`docs/process/`	`npm run process:docs`
`docs/visual/platform-data.js`	`npm run visual`
`docs/deployment/outstanding-rollup.json`	`npm run outstanding`
Identification packages	`npm run idphase3`, `npm run idphase4`
E-mail bodies in packages	`npm run emailcatalogue`
Estate Console	`npm run console`

The failing check always names the command.

7.10 Debugging a live problem

Collect evidence before changing anything:

1. **Did the request reach the flow?** Read the flow's run history (or `api.flow.microsoft.com` runs). No run means the browser never sent it, or the gateway refused it.
2. **What did the flow receive and answer?** The run's trigger outputs (headers, body) and each action's inputs and outputs, from the run record. The host `…environment.api.powerplatformusercontent.com` must be reachable.
3. **What did the browser record?** In the browser console: `(await import('/core/state.js')).State.get().runtime.lastLoad` holds the last load's outcome and message.

4. **Reproduce with the real code.** Run the deployed site's own files in Playwright, replacing only the flow calls with recorded replies, then repeat with real calls. One variable at a time.

A worked example of this method is in Troubleshooting §10.2.